Jittor 代码复现：DAFormer

域自适应语义分割的系统优化方案

南开大学 计算机科学与技术

2023级 付炘浩

目录

南開大學
Nankai University

01 论文梳理

DAFormer: Improving Network Architectures and Training Strategies for Domain-Adaptive Semantic Segmentation（CVPR 2022）

01 论文梳理

1.1 引入：恶劣环境语义分割的挑战

雾

夜

雨

雪

环境复杂

雾、夜、雨、雪等导致能见度低、对比度弱。

感知瓶颈

传统模型过度依赖局部纹理，在信息残缺时极易失效。

标注困境

恶劣环境下的像素级标注成本极高，甚至不可获得。

⬇

域自适应语义分割

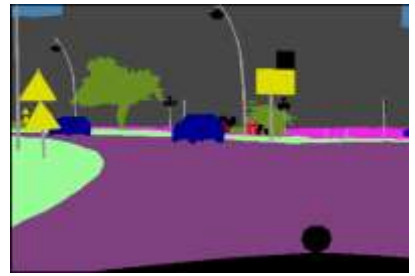
01 论文梳理

1.2 无监督域自适应（Unsupervised Domain Adaptation，UDA）


源域

Domain Shift

目标域

动机

域偏移和数据标注成本的差异。

核心目的

无需目标域数据标签，通过适配策略让模型从源域迁移到目标域。

研究方向

- 域自适应策略：对抗训练、自训练
- 网络架构设计：强表征能力与域鲁棒性
- 训练优化机制：解决过拟合等问题

01 论文梳理

1.3 自训练机制与数据增强策略

自训练机制：通过师生协同缩小域偏移，无需目标域标注

$$L_{seg} = L_S + L_T$$

语义分割损失

$$\phi_{t+1} \leftarrow \alpha\phi_t + (1-\alpha)\theta_t$$

教师模型的 EMA 更新（θ_t：学生模型的权重）

Teacher：为目标域生成伪标签 + 置信度估计

Student：同时学习源域真实标签与目标域高质量伪标签

数据增强策略：学习域无关的通用特征

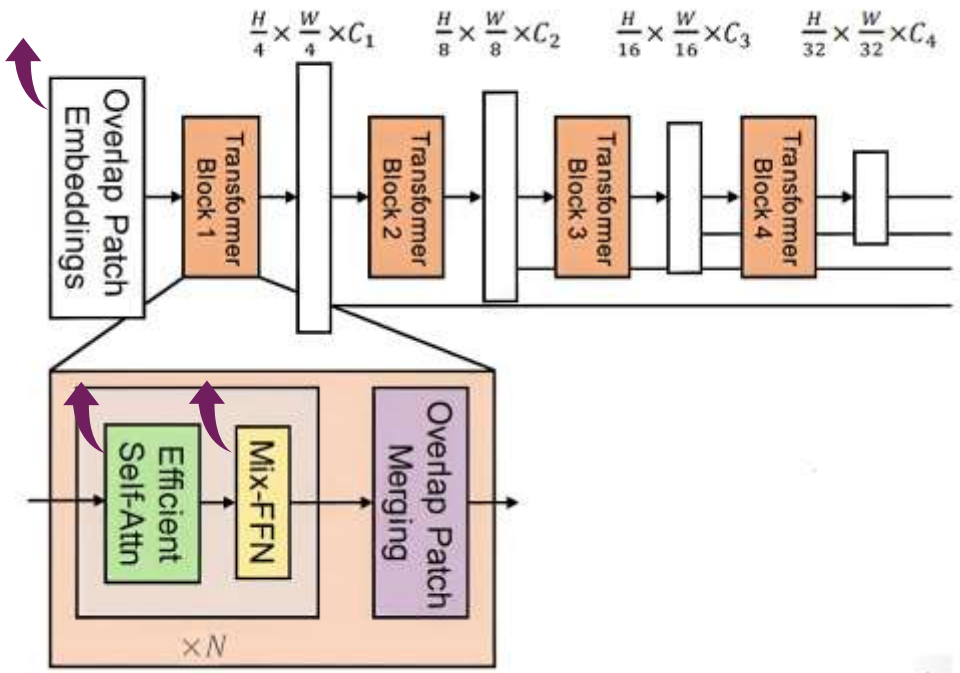
a. 跨域混合采样：源域、目标域同时采样，利用 ClassMix 混合

b. 基础增强：颜色抖动、高斯模糊

ClassMix 示意

01 论文梳理

1.4 网络架构 —— MiT-b5 编码器



摘自 SegFormer

$Step1$ 下采样：重叠补丁合并，使用重叠分块保留空间连续性

$Step2$ 自注意力模块：序列缩减，降低计算复杂度

$$N' = \frac{H \times W}{sr\_ratio^2}$$

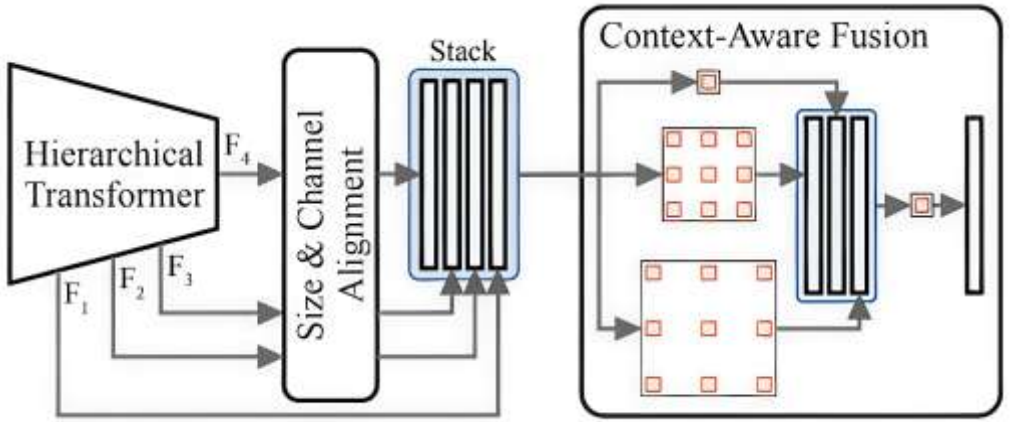
$$Attn(Q,K,V) = Softmax\left(\frac{Q \cdot K^T}{\sqrt{d_h}}\right) \cdot V$$

$Step3$ Mix-FFN：隐式位置编码，支持测试与训练分辨率不一致

$$\text{Mix-FFN}(x) = \text{Dropout}\left(\text{Linear}_2\left(\text{Dropout}\left(\text{GELU}\left(\text{DWConv}(\text{Linear}_1(x))\right)\right)\right)\right)$$

01 论文梳理

1.4 网络架构 —— 多级上下文感知特征融合解码器



$Step1$ 嵌入层统一通道维度

$$F_i' = \mathrm{EmbedLayer}_i(F_i)$$

$Step2$ 上采样对齐空间尺寸

$$F_i'' = \mathrm{Resize}\left(F_i', \mathrm{size}(F_1)\right)$$

$Step3$ Sep-ASPP 融合上下文

$$F_{\mathrm{out}} = \mathrm{SepASPP}\left(\mathrm{Concat}(F_1'', F_2'', F_3'', F_4'')\right)$$

01 论文梳理

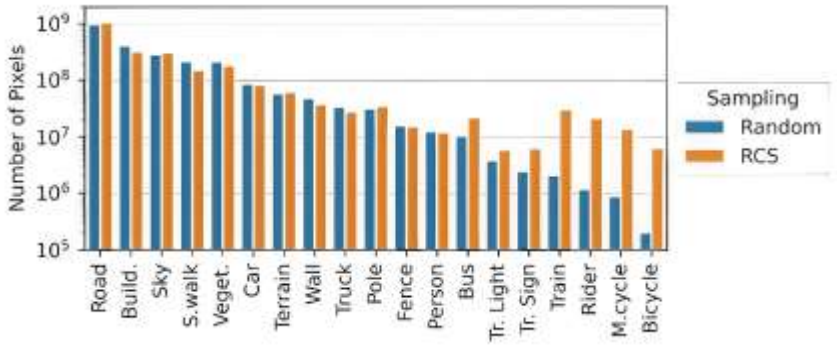
1.5 DAFormer 训练策略

学习率预热：保留 ImageNet 预训练特征，稳定梯度。

在预热阶段，第 t 步迭代的学习率设置为：

$$\eta_t = \eta_{\text{base}} \cdot \frac{t}{t_{\text{warm}}}$$

源域稀有类采样策略（RCS）



GTA 数据集的采样结果

动机：源域数据存在长尾分布，迭代中会阻碍模型对稀有类别的学习。

$$P(c) = \frac{e^{(1-f_c)/T}}{\sum_{c'=1}^{C} e^{(1-f_{c'})/T}}$$

某一类别 c 的采样概率

01 论文梳理

目标类 ImageNet 特征距离策略（FD）：保留通用特征，抑制源域过拟合

$$\mathcal{L}_{\mathcal{FD}}^{(i)} = \frac{\sum_{j=1}^{H_F \times W_F} d^{(i,j)} \cdot M_{\text{things}}^{(i,j)}}{\sum_j M_{\text{things}}^{(i,j)}}$$

特征距离（FD）损失

$$M_{\text{things}}^{(i,j)} = \sum_{c'=1}^{C} y_{S,\text{small}}^{(i,j,c')} \cdot \left[c' \in C_{\text{things}}\right]$$

二值掩码

$$d^{(i,j)} = |F_{\text{ImageNet}}\left(x_S^{(i)}\right)^{(j)} - F_\theta\left(x_S^{(i)}\right)^{(j)}|_2$$

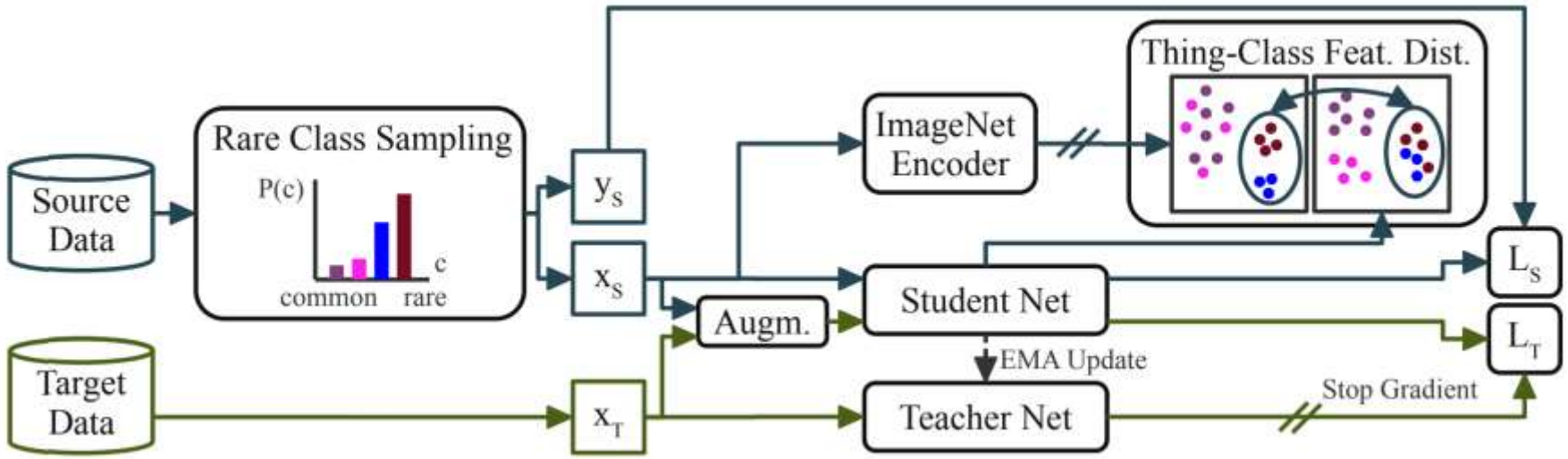
特征向量 $d(i,j)$

$$\mathcal{L} = \mathcal{L}_S + \mathcal{L}_T + \lambda_{FD}\mathcal{L}_{\mathcal{FD}}$$

DAFormer 总损失

01 论文梳理

1.6 小结



DAFormer 的完整流程

1. DAFormer 聚焦 UDA，针对合成 - 真实数据域偏移、真实数据标注成本高的痛点，突破传统方法依赖过时 CNN 架构的局限。
2. 创新点：Transformer 架构 + 三重训练策略。MiT-B5 编码器 + 多尺度融合解码器奠定基础，RCS、FD 与学习率预热解决过拟合与训练稳定性问题。

02 代码介绍

02 代码介绍

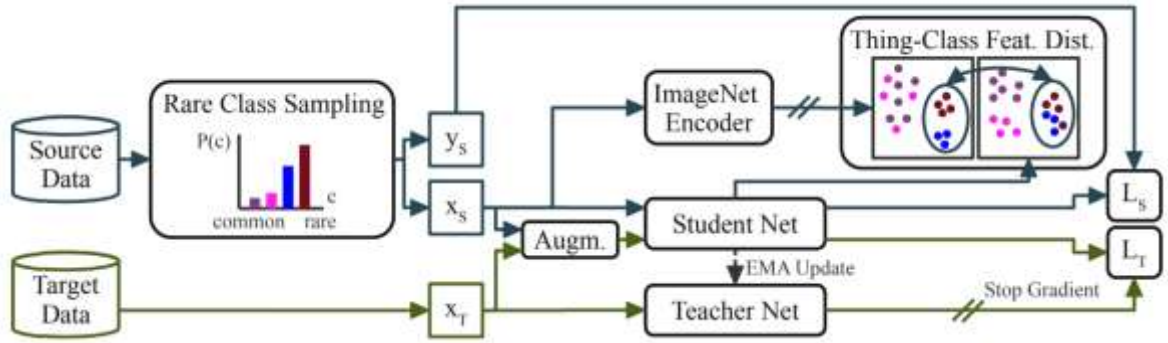
2.1 文件结构

```
Jittor_DAFormer
├─configs
│      gta2cs_daformer.py          # 实验配置
├─data                             # 图片数据
│      ...
├─datasets                         # 制作数据集
│      base.py                     # 基类
│      cityscapes.py               # Cityscrapes 数据集类
│      gta.py                      # GTA 数据集类
│      uda_dataset.py              # UDA 数据集类
├─models
│   │  ema.py                      # EMA 模型封装
│   │  segmentor.py                # 完整的学生模型
│   ├─backbones
│   │      mit_b5.py               # Backbone: MiT-B5
│   └─decode_heads
│          daformer_head.py        # 解码器
├─pretrained
│      mit_b5.pth                  # MiT ImageNet 权重（b5）
├─trainer
│      train_daformer.py           # DAFormer 单步训练流程
├─utils
│   │  checkpoint.py               # checkpoint 辅助函数
│   │  logger.py                   # 日志记录工具
│   │  losses.py                   # 特征距离计算
│   │  lr_scheduler.py             # 学习率调度器
│   │  mix.py                      # 类别混合增强
│   │  pseudo_label.py             # 伪标签生成
│   │  transform.py                # 图像变换（制作数据集用）
│   │  plot_logs.py                # 绘制训练日志图像
│   └─convert_datasets             # 数据预处理
│          cityscapes.py
│          gta.py
└─train.py                         # 训练核心文件
```

02 代码介绍

2.2 理解 DAFormer 完整流程

```
# 执行一步 UDA 训练
def train_step(self, batch, iteration):
    self.model.train()
    # 1. 源域监督前向与反向
    src_losses, src_feat = self._train_on_source(
        batch['img’],
        batch['gt_semantic_seg'])
    src_loss = src_losses['loss’]
    # Jittor: optimizer.backward(loss)
    self.optimizer.backward(src_loss)
    # 2. 特征距离损失（可选）
    if self.enable_fdist and src_feat is not None:
        fdist_loss = self._calc_feat_dist(
            batch['img’],
            batch['gt_semantic_seg’],
            src_feat)
        src_loss = src_loss + fdist_loss
        self.optimizer.backward(fdist_loss)
    # 3. 伪标签生成（教师预测， no_grad）
    with jt.no_grad():
        pseudo_label, pseudo_weight =
        self._generate_pseudo_label(batch['target_img'])
```




```
    # 4. ClassMix 增强并在混合样本上训练
    mixed_img, mixed_lbl, mixed_weight =
self._apply_class_mix(
        batch['img'], batch['gt_semantic_seg'],
        batch['target_img'], pseudo_label, pseudo_weight
    )
    mix_losses = self._train_on_mixed(
        mixed_img, mixed_lbl, mixed_weight)
    mix_loss = mix_losses['loss']
    self.optimizer.backward(mix_loss)
    # 5. 优化器步进、EMA 更新与学习率调度
    self.optimizer.step()
    self.ema_model.update(iteration, self.model)
    self.lr_schedule.step()
    # 返回总损失（用于上层计数或打印）
    return {‘total_loss’: (src_losses[‘loss’] +
mix_loss).item()}
```

trainer\train_daformer.py

02 代码介绍

2.3 数据增强策略 —— DACS

```python
# 强增强变换函数
def strong_transform(param, data=None, target=None):
    # 1. 类别混合
    data, target = one_mix(mask=param['mix'], data=data, target=target)
    # 2. 颜色抖动：Jittor 中没有 kornia 库，需要自行实现
    data, target = color_jitter(color_jitter=param['color_jitter’], s=param['color_jitter_s’], p=param['color_jitter_p'],
                                mean=param['mean’] ,std=param['std’] ,data=data, target=target)
    # 3. 高斯模糊：同样需要自行实现
    data, target = gaussian_blur(blur=param['blur'], data=data, target=target)
    return data, target

# 进行一次类别混合
def one_mix(mask, data=None, target=None):
    if mask is None:
        return data, target
    # mask: [1,1,H,W] -> 取出 [H,W]
    mask_2d = mask[0, 0]  # [H,W]
    if data is not None:
        # data: [2,3,H,W]
        mask_3d = mask_2d.unsqueeze(0)  # [1,H,W]
        data = (mask_3d * data[0] + (1 - mask_3d) * data[1]).unsqueeze(0)  # [1,3,H,W]
    if target is not None:
        # target: [2,H,W]
        target = (mask_2d * target[0] + (1 - mask_2d) * target[1]).unsqueeze(0)  # [1,H,W]
    return data, target
```

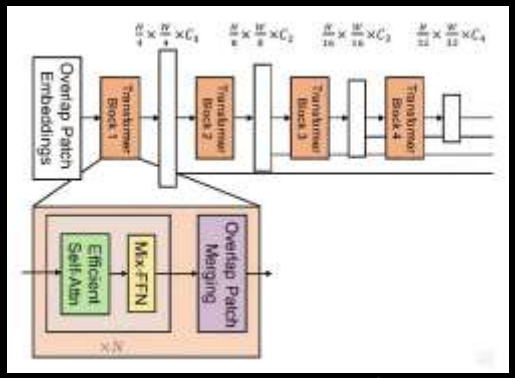
utils\mix.py



02 代码介绍

2.4 网络架构 —— MiT-b5 编码器

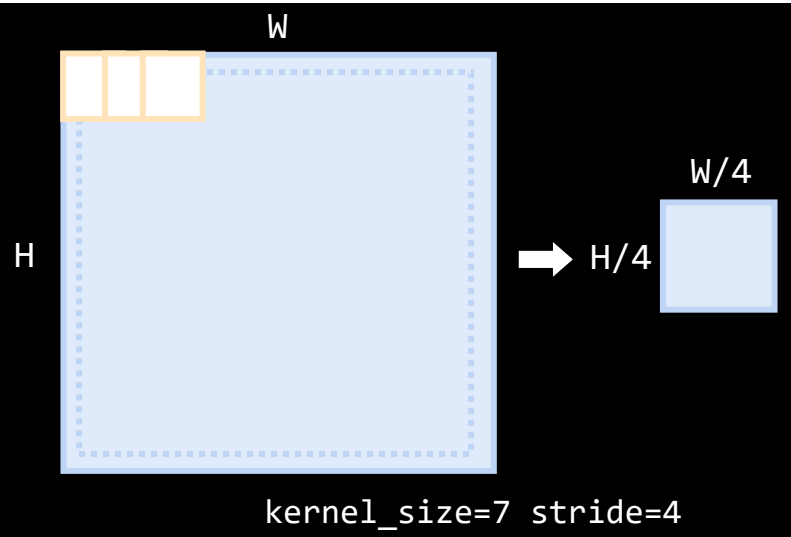
```python
# MixVisionTransformer (MiT-B5) 核心：4-stage 多尺度输出，embed_dims=[64,128,320,512]
class MixVisionTransformer(nn.Module):
    def __init__(self, in_chans=3, embed_dims=[64,128,320,512], num_heads=[1,2,5,8],mlp_ratios=[4,4,4,4], qkv_bias=False,
                 qk_scale=None, drop_rate=0., attn_drop_rate=0.,drop_path_rate=0.1, norm_layer=nn.LayerNorm, depths=[3,6,40,3],
                 sr_ratios=[8,4,2,1],pretrained=None, freeze_patch_embed=False):
        super().__init__()
        # Patch embeddings (重叠卷积)
        self.patch_embed1 = OverlapPatchEmbed(patch_size=7,stride=4,in_chans=in_chans,embed_dim=embed_dims[0])
        self.patch_embed2 = OverlapPatchEmbed(patch_size=3,stride=2,in_chans=embed_dims[0],embed_dims=embed_dims[1])
        # ··· 同理构建 patch_embed3 和 patch_embed4，patch_size、stride 同 patch_embed2，略
        dpr = [x.item() for x in jt.linspace(0, drop_path_rate, sum(depths))] # stochastic depth ratios
        # blocks per stage + norms
        cur = 0
        self.block1 = nn.ModuleList([Block(embed_dims[0], num_heads[0], mlp_ratio=mlp_ratios[0],qkv_bias=qkv_bias,
                     qk_scale=qk_scale, drop=drop_rate,attn_drop=attn_drop_rate, drop_path=dpr[cur+i], sr_ratio=sr_ratios[0])
                     for i in range(depths[0])])
        self.norm1 = norm_layer(embed_dims[0]); cur += depths[0]
        # ··· 同理构建 block2/block3/block4 和 norm2/norm3/norm4，略
    def execute(self, x):
        B = x.shape[0]; outs = []
        x, H, W = self.patch_embed1(x)
        for blk in self.block1: x = blk(x, H, W)
        x = self.norm1(x).reshape(B, H, W, -1).permute(0,3,1,2); outs.append(x)
        # ··· 同理执行 block2/block3/block4，得到 4 个尺度的特征图，略
        return outs
```



models\backbones\mit_b5.py

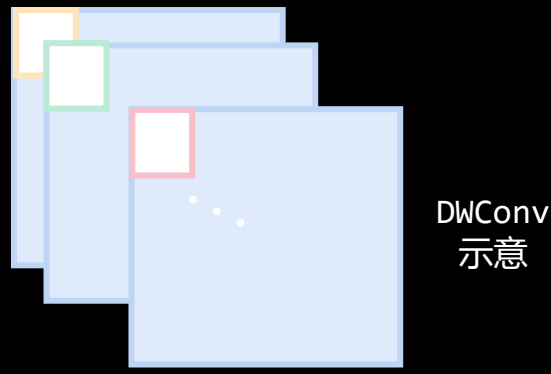
02 代码介绍

```python
# Overlap Patch Embeddings
class OverlapPatchEmbed(nn.Module):
    def __init__(self, patch_size, stride, in_chans, embed_dim):
        super().__init__()
        self.proj = nn.Conv2d(
            in_chans,
            embed_dim,
            kernel_size=patch_size,
            stride=stride,
            padding=(patch_size // 2, patch_size // 2)
        )
        self.norm = nn.LayerNorm(embed_dim)
    def execute(self, x):
        # ··· x -> 卷积层 ->归一化层（略）
```



models\backbones\mit_b5.py

```python
# Mix-FFN
class Mlp(nn.Module):
    def __init__(self, in_features,hidden_features=None, out_features=None,
                 act_layer=nn.GELU, drop=0.):
        super().__init__()
        self.fc1 = nn.Linear(in_features, hidden_features)
        self.dwconv = DWConv(hidden_features) # 深度可分离卷积
        self.act = act_layer()
        self.fc2 = nn.Linear(hidden_features, out_features)
        self.drop = nn.Dropout(drop)
    def execute(self, x, H, W):
        # ··· x -> fc1 -> dwconv -> act -> fc2 -> drop  （略）
```



models\backbones\mit_b5.py

02 代码介绍

```python
# Efficient Self-Attn
class Attention(nn.Module):
    def __init__(self, dim, num_heads=8, qkv_bias=False, qk_scale=None, attn_drop=0., proj_drop=0., sr_ratio=1):
        super().__init__()
        # ...
        self.q = nn.Linear(dim, dim, bias=qkv_bias)
        self.kv = nn.Linear(dim, dim * 2, bias=qkv_bias)
        # ...
        self.sr_ratio = sr_ratio # 序列缩减
        if sr_ratio > 1:
            self.sr = nn.Conv2d(dim, dim, kernel_size=sr_ratio, stride=sr_ratio)
            self.norm = nn.LayerNorm(dim)
    def execute(self, x, H, W):
        B, N, C = x.shape
        # q: (B, num_heads, N, C//num_heads)
        q = self.q(x).reshape(B, N, self.num_heads, C // self.num_heads).permute(0, 2, 1, 3)
        if self.sr_ratio > 1: # 序列缩减策略
            x_ = x.permute(0, 2, 1).reshape(B, C, H, W) # (B, C, H, W)
            x_ = self.sr(x_).reshape(B, C, -1).permute(0, 2, 1)  # (B, N', C)
            x_ = self.norm(x_) # (B, N', C)
            # (2, B, num_heads, N', C//num_heads)
            kv = self.kv(x_).reshape(B, -1, 2, self.num_heads, C // self.num_heads).permute(2, 0, 3, 1, 4)
        else:
            # (2, B, num_heads, N, C//num_heads)
            kv = self.kv(x).reshape(B, -1, 2, self.num_heads, C // self.num_heads).permute(2, 0, 3, 1, 4)
        k, v = kv[0], kv[1] # (B, num_heads, N(or N') , C//num_heads)
```

models\backbones\mit_b5.py

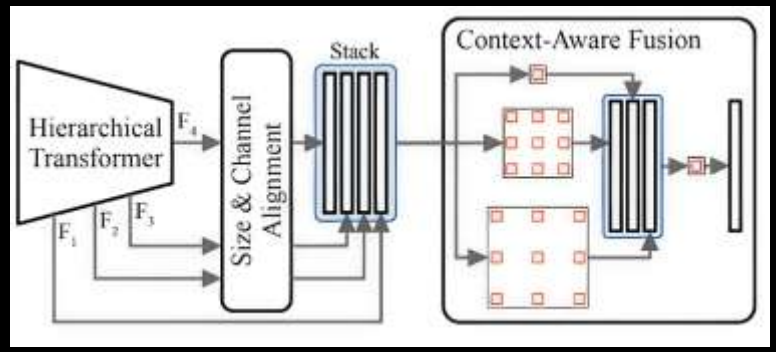
$$Attn(Q,K,V) = Softmax\left(\frac{Q \cdot K^T}{\sqrt{d_h}}\right) \cdot V$$

02 代码介绍

2.5 网络架构 —— 多级上下文感知特征融合解码器

```python
class DAFormerHead(nn.Module):
    def __init__(self, in_channels, in_index, channels=256, dropout_ratio=0.1, num_classes=19, norm_cfg=dict(type='BN'),
                 align_corners=False, decoder_params=None):
        super().__init__()
        # ··· 解析参数（略）
        embed_layers = [] # ··· 为每个输入尺度构建对应的嵌入层（略）
        self.embed_layers = nn.ModuleList(embed_layers)
        # 融合层：使用 Sep-ASPP 融合多尺度特征
        self.fuse_layer = build_layer(sum(embed_dims), self.channels, **fusion_cfg)
        # Dropout + 分类头
        self.dropout = nn.Dropout2d(dropout_ratio) if dropout_ratio > 0 else None
        self.conv_seg = nn.Conv2d(channels, num_classes, kernel_size=1)
    def execute(self, inputs):
        # ··· 收集 x、n、os_size （略）
        _c = {}
        for idx, i in enumerate(self.in_index):
            _c[i] = self.embed_layers[idx](x[i]) # 统一通道数：(B, H*W, embed_dim)
            _c[i] = _c[i].permute(0, 2, 1).reshape(n, -1, x[i].shape[2], x[i].shape[3])  # (B, C, H, W)
            # 统一尺寸：上采样到最高分辨率
            if _c[i].shape[2:] != os_size:
                _c[i] = resize(_c[i], size=os_size,mode='bilinear', align_corners=self.align_corners)
        x = jt.concat(list(_c.values()), dim=1)  # 拼接所有尺度的特征 (B, sum(embed_dims), H/4, W/4)
        x = self.fuse_layer(x)  # 使用 Sep-ASPP 融合 (B, channels, H/4, W/4)
        # Dropout + 分类
        x = self.dropout(x) if self.dropout is not None else x
        x = self.conv_seg(x)  # (B, num_classes, H/4, W/4)
        return x
```

models\decode_heads\daformer_head.py



02 代码介绍

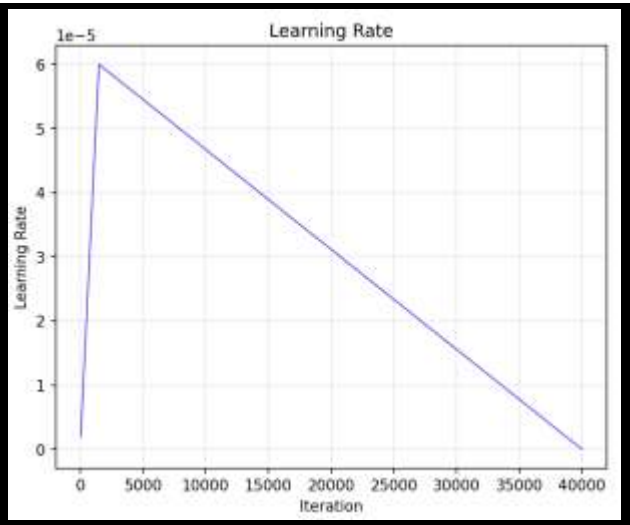
```python
# 深度可分离 ASPP 模块
class ASPPWrapper(nn.Module):
    def __init__(self, in_channels, channels,
                 sep, dilations, pool,norm_cfg,
                 act_cfg,align_corners):
        super().__init__()
        # 深度可分离 ASPP 模块
        self.aspp_modules =  DepthwiseSeparableASPPModule
            (dilations=dilations, in_channels=in_channels,
            channels=channels, norm_cfg=norm_cfg,
            act_cfg=act_cfg)
        # Bottleneck: 融合所有 ASPP 分支
        self.bottleneck = ConvModule(
            len(dilations) * channels, channels,
            kernel_size=3, padding=1,
            norm_cfg=norm_cfg, act_cfg=act_cfg)
    def execute(self, x):
        aspp_outs = []
        # ASPP 分支
        aspp_outs.extend(self.aspp_modules(x))
        # 拼接并融合
        aspp_outs = jt.concat(aspp_outs, dim=1)
        output = self.bottleneck(aspp_outs)
        return output    models\decode_heads\daformer_head.py
```

```python
# 深度可分离 ASPP 模块的多个分支
class DepthwiseSeparableASPPModule(nn.ModuleList):
    def __init__(self, dilations, in_channels, channels,
                 norm_cfg, act_cfg):
        super().__init__()
        self.dilations = dilations
        for dilation in dilations:
            if dilation == 1:
                # dilation=1 时使用标准 1x1 卷积
                self.append(ConvModule(
                        in_channels, channels, kernel_size=1,
                        norm_cfg=norm_cfg, act_cfg=act_cfg))
            else:
                # dilation>1 时使用深度可分离卷积
                self.append(
                    DepthwiseSeparableConvModule(
                        in_channels, channels, kernel_size=3,
                        dilation=dilation, padding=dilation,
                        norm_cfg=norm_cfg, act_cfg=act_cfg))
    def execute(self, x):
        aspp_outs = []
        for aspp_module in self:
            aspp_outs.append(aspp_module(x))
        return aspp_outs    models\decode_heads\daformer_head.py
```

02 代码介绍

2.6 DAFormer 训练策略

```python
# 学习率预热策略：多项式学习率调度器
class PolyLRWithWarmup:
    def __init__(self, optimizer, max_iters, warmup_iters=1500,
                 warmup_ratio=1e-6, power=1.0, min_lr=0.0):
        # ··· 解析参数并保存初始学习率（略）
    def step(self):
        self.last_iter += 1
        # 为每个参数组计算新的学习率
        for i, param_group in enumerate(self.optimizer.param_groups):
            base_lr = self.base_lrs[i]
            new_lr = self._compute_lr(base_lr, self.last_iter)
            param_group['lr'] = new_lr
    def _compute_lr(self, base_lr, current_iter):
        if current_iter < self.warmup_iters:
            # Warmup 阶段：线性增长
            k = (1 - self.warmup_ratio) * current_iter / self.warmup_iters + self.warmup_ratio
            lr = base_lr * k
        else:
            # Poly 衰减阶段
            progress = (current_iter - self.warmup_iters) / (self.max_iters - self.warmup_iters)
            lr = (base_lr - self.min_lr) * pow(1 - progress, self.power) + self.min_lr
            # 确保学习率不低于最小值
            lr = max(lr, self.min_lr)
        return lr
    # ··· 省略 state_dict 和 load_state_dict
```



$$lr(t) = lr_{\text{base}} \cdot \left[r_{\text{warm}} + (1 - r_{\text{warm}}) \cdot \frac{t}{T_{\text{warm}}} \right]$$

utils\lr_scheduler.py

02 代码介绍

```python
class UDADataset(Dataset): # RCS 策略
    def __init__(self, source, target, rare_class_sampling=None,
                 batch_size=2, num_workers=4, shuffle=True):
        super().__init__()
        # ··· 解析参数（略）
        self._init_rcs() if self.rcs_enabled else None
    def _init_rcs(self):
        self.rcs_classes, self.rcs_classprob = get_rcs_class_probs(
            self.source.data_root, self.rcs_class_temp)
        # ··· 过滤：只保留 像素数 > min_pixels 的样本（略）
    def get_rare_class_sample(self):
        c = np.random.choice(self.rcs_classes, p=self.rcs_classprob)
        filename = np.random.choice(self.samples_with_class[str(c)])
        source_idx = self.file_to_idx[filename]
        source_sample = self.source[source_idx]
        if self.rcs_min_crop_ratio > 0:
            for attempt in range(10):
                gt_seg = source_sample['gt_semantic_seg']
                if isinstance(gt_seg, jt.Var):
                    n_class = jt.sum(gt_seg == c).item()
                else:
                    n_class = np.sum(gt_seg == c)
                if n_class > self.rcs_min_pixels * self.rcs_min_crop_ratio:
                    break
                source_sample = self.source[source_idx]
        target_idx = np.random.randint(0, len(self.target))
        target_sample = self.target[target_idx]
        return {**source_sample,  'target_img': target_sample['img']}
    # ··· _len__、__getitem__ 的方法实现（略）
```

datasets\uda_dataset.py

$$P(c) = \frac{e^{(1-f_c)/T}}{\sum_{c'=1}^{C} e^{(1-f_{c'})/T}}$$

```python
# 计算 RCS 的类别概率
def get_rcs_class_probs(data_root, temperature):
    with open(osp.join(data_root, 'samples_with_class.json'),
'r') as f:
        samples_with_class = json.load(f) # 读取反向索引
    # 统计每个类别的总像素数
    overall_class_stats = {}
    for class_id, file_list in samples_with_class.items():
        class_id = int(class_id)
        total_pixels = sum(n for _, n in file_list)
        overall_class_stats[class_id] = total_pixels
    # 按像素数排序（从少到多）
    overall_class_stats = {
        k: v for k, v in sorted(
            overall_class_stats.items(),
            key=lambda item: item[1]
        )
    }
    classes_list = list(overall_class_stats.keys())
    pixels_list = list(overall_class_stats.values())
    # 使用 numpy float64 避免精度丢失
    freq = np.array(pixels_list, dtype=np.float64)
    freq = freq / freq.sum()
    freq = 1.0 - freq
    # 转回 Jittor 做 softmax
    freq = jt.array(freq).float32()
    freq = jt.nn.softmax(freq / temperature, dim=-1)
    return classes_list, freq.numpy()
```

datasets\uda_dataset.py

02 代码介绍

```python
# FD 策略：计算 thing class 上的特征距离
def masked_feature_distance(student_feat, teacher_feat, gt_seg,
                            mask_classes=None, scale_min_ratio=0.75,
                            num_classes=19, ignore_index=255):
    scale_factor = gt_seg.shape[-1] // student_feat.shape[-1]
    # 将标签下采样到特征图尺寸 [B, 1, H/32, W/32]
    gt_rescaled = downscale_label_ratio(
            gt_seg,
            scale_factor=scale_factor,
            min_ratio=scale_min_ratio,
            n_classes=num_classes,
            ignore_index=ignore_index
    ).int64().stop_grad()
    # 创建掩码：仅保留 mask_classes 中的类别 [B, 1, H/32, W/32]
    fdclasses = jt.array(mask_classes).int64()
    fdist_mask = jt.any(gt_rescaled[..., None] == fdclasses, dim=-1)
    # 如果掩码区域为空，返回 0
    if int(fdist_mask.sum().data[0]) == 0:
        return jt.array(0.0)
    # 计算特征距离（L2 范数）
    feat_diff = student_feat - teacher_feat  # [B, C, H/32, W/32]
    pw_feat_dist = jt.norm(feat_diff, p=2, dim=1)  # [B, H/32, W/32]
    # 应用掩码 - Jittor 中使用 where 提取s
    fdist_mask_squeezed = fdist_mask.squeeze(1)  # [B, H/32, W/32]
    masked_pw_feat_dist = pw_feat_dist * fdist_mask_squeezed.float32()
    # 计算平均特征距离（只统计掩码区域）
    return masked_pw_feat_dist.sum() / fdist_mask_squeezed.sum()
```

$$y^{c}_{S,\text{small}} = [\text{AvgPool}(y^{c}_{S}, H/H_F, W/W_F) > r]$$

$$M^{(i,j)}_{\text{things}} = \sum_{c'=1}^{C} y^{(i,j,c')}_{S,\text{small}} \cdot \left[c' \in C_{\text{things}}\right]$$

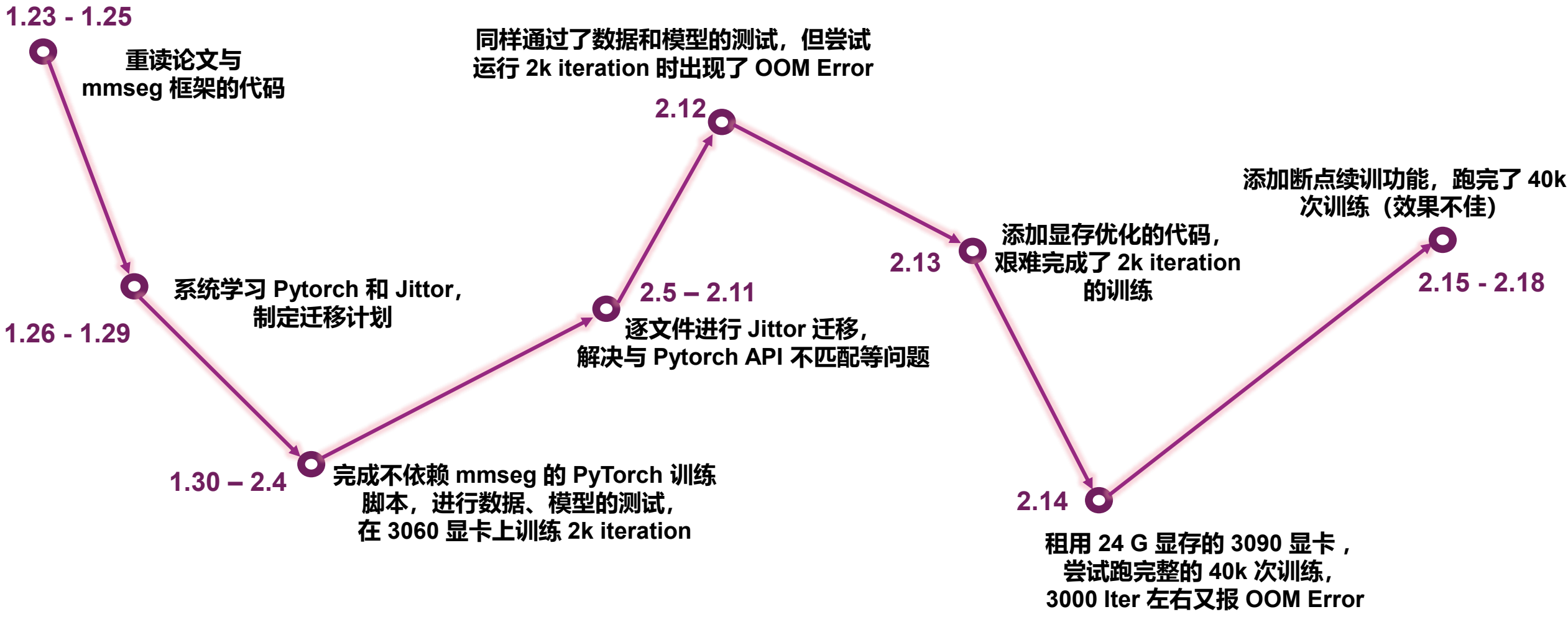
```python
# 下采样
def downscale_label_ratio(gt, scale_factor, min_ratio,
n_classes, ignore_index=255):
    ignore_substitute = n_classes # 用于忽略标签的替代值
    out = gt.clone() # 克隆避免修改原始数据
    out[out == ignore_index] = ignore_substitute
    # One-hot 编码 (B,n_classes+1,H,W)
    out_squeezed = out.squeeze(1)
    out = jt.nn.one_hot(out_squeezed.int32(),
num_classes=n_classes + 1).permute(0, 3, 1, 2)
    # 平均池化下采样 (B, n_classes+1, H/scale, W/scale)
    out = jt.nn.avg_pool2d(out.float32(),
kernel_size=scale_factor)
    # gt_ratio: 每个位置的最大占比值 [B,1,H,W]
    gt_ratio = jt.max(out, dim=1, keepdims=True)
    # out: 每个位置的主导类别索引 [B,1,H,W]
    out = jt.argmax(out, dim=1, keepdims=True)[0]
    # 恢复 ignore_index
    out[out == ignore_substitute] = ignore_index
    # 占比低于阈值的区域设为 ignore_index
    out[gt_ratio < min_ratio] = ignore_index
    return out
```

utils\losses.py

03 实施细节

03 实施细节

3.1 迁移历程



03 实施细节

3.2 迁移遇到的问题

```
freq = torch.tensor(list(overall_class_stats.values()), dtype=torch.float32)
freq = freq / torch.sum(freq)
freq = 1 - freq
freq = torch.softmax(freq / temperature, dim=-1)
```

Pytorch

```
# 使用 numpy float64 避免精度丢失
freq = np.array(pixels_list, dtype=np.float64)
freq = freq / freq.sum()
freq = 1.0 - freq
# 转回 Jittor 做 softmax
freq = jt.array(freq).float32()
freq = jt.nn.softmax(freq / temperature, dim=-1)
```

Jittor

RCS 计算采样概率

精度问题

Jittor 的 float32 在处理超过 10 亿的像素数时精度不足，导致除法出错。

```
RuntimeError: [f 0219 16:29:50.423641 00 executor.cc:686]
Execute fused operator(141/9869) failed.
[JIT Source]: /root/.cache/jittor/jt1.3.10/g++9.4.0/py3.8.19/Linux-5.19.0-5x09/IntelRXeonRCPUx65/8fe3/default/cu12.2.140_sm_86
M_4__YDIM_7__OVERFLOW_itof_0x0___INDEX0_i0__INDEX1_i2__IN___hash_5050cb521a678353_op.cc
[OP TYPE]: fused_op:( reindex, broadcast_to, binary.multiply, reduce.add,)
[Input]: float32[1,1024,96,96,], float32[1,256,96,96,],
[Output]: float32[256,1024,3,3,],
[Async Backtrace]: not found, please set env JT_SYNC=1, trace_py_var=3
[Reason]: [f 0219 16:29:50.422992 00 cuda_device_allocator.cc:31] Unable to alloc cuda device memory for size 143917056
```

OOM Error

显存爆炸问题

可能是 Jittor 的异步执行、计算图管理机制、算子融合策略导致。

03 实施细节

	PyTorch	Jittor	
张量属性：shape/size	tuple.size() 表示各维度大小，等价于 .shape	. tuple.size() 返回元素总数	不用 .size() 获取维度，用 .shape
argmax	argmax(dim) 直接返回 Tensor	argmax(dim) 返回 (indices, values)	取第一个元素要用 argmax(dim)[0]
ModuleDict / ModuleList	有 ModuleDict 和 ModuleList	无 ModuleDict，只能用 ModuleList	解码头的 embed_layers 用 ModuleList 收集
参数冻结	param.requires_grad = Falseparam.detach_()	param.stop_grad()	Jittor 没有 .requires_grad
DataLoader	需显式构建 DataLoader	内置数据加载系统或自定义迭代器	写 dataloader 方式不同
梯度清零	需 optimizer.zero_grad()	不需要手动清零	Jittor 自动管理梯度清零
反向传播	loss.backward()	没有 .backward() 方法，必须 optimizer.backward(loss)	所有训练循环必须改写

逐文件迁移过程中遇到的 Jittor / Pytorch 差异及解决方案

03 实施细节

3.3 训练与测试流程

1. 安装依赖+CUDA + cuDNN

```
pip install -r requirements.txt
python3.8 -m jittor_utils.install_cuda
```

2. 数据预处理，得到 TrainIds 标签图 + 类别统计 json 文件

```
python utils/convert_datasets/cityscapes.py data/cityscapes --nproc 8
python utils/convert_datasets/gta.py data/gta --nproc 8
```

3. 开始训练（支持 -- resume 参数断点续训）

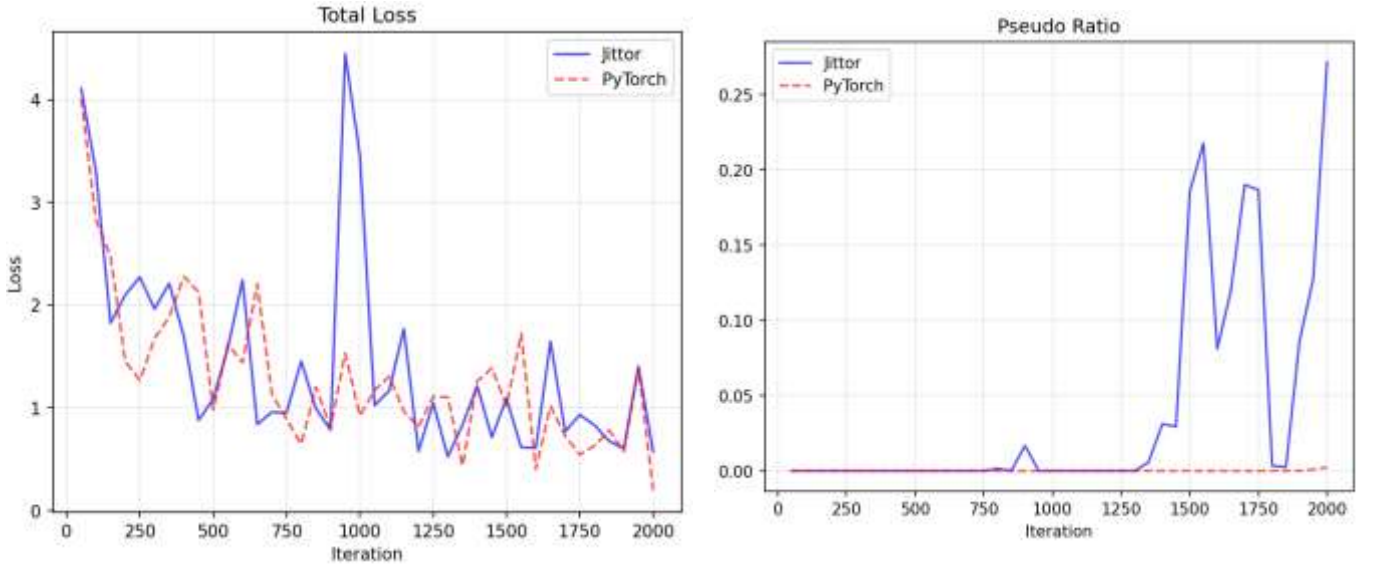
```
python train.py --config configs/gta2cs_daformer.py
```

4. 推理测试

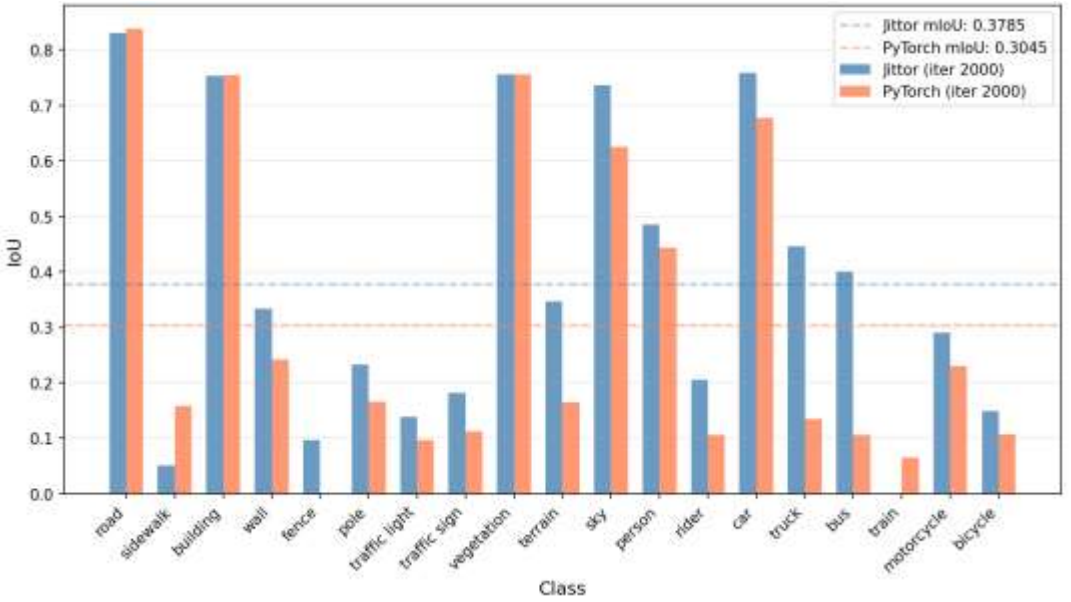
```
python -m demo.image_demo demo/demo.png ***.pth
```

03 实施细节

3.4 结果评估　3.4.1 Jittor 和 Pytorch 训练 2k Iter 的结果对比 —— 整体是对齐的



训练过程对比



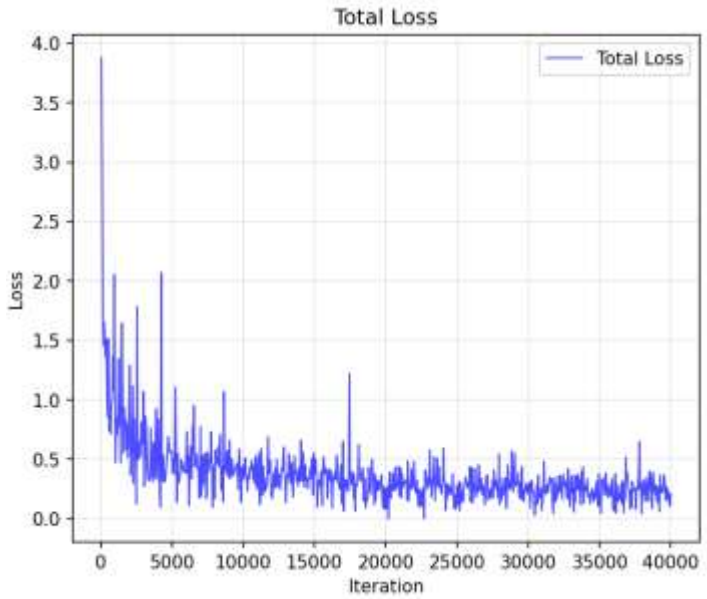
IoU 对比

Jittor 框架下训练 loss 抖动较严重，原因可能是 Jittor 的某些数值操作稳定性差；并且伪标签激活要早于 Pytorch。

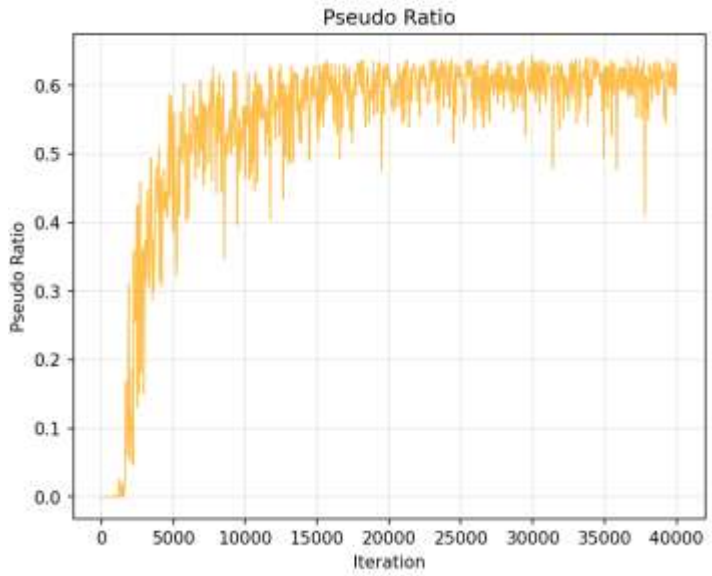
Jittor 的语义分割效果却优于 Pytorch，原因可能是伪标签激活早，更快适应目标域，也可能是某些数值误差意外提高了模型性能。

03 实施细节

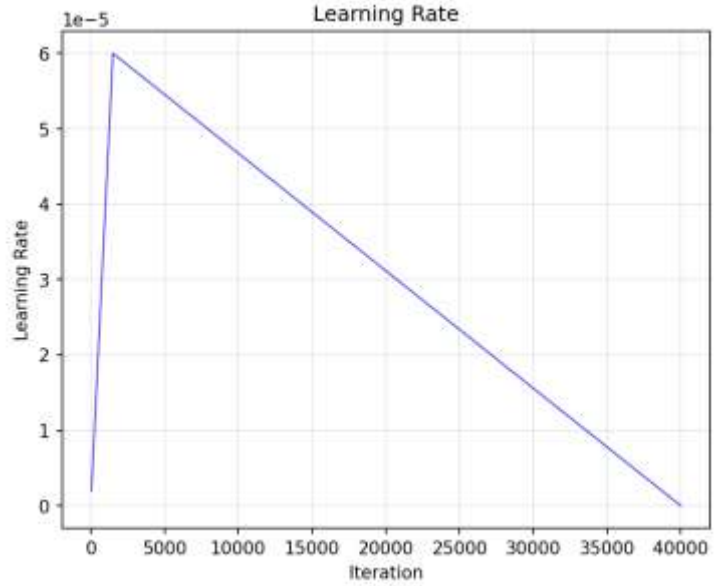
3.4.2 Jittor 完整训练 40k Iter 的结果评估（依赖断点续训，1/4 数据集）



Total Loss 曲线



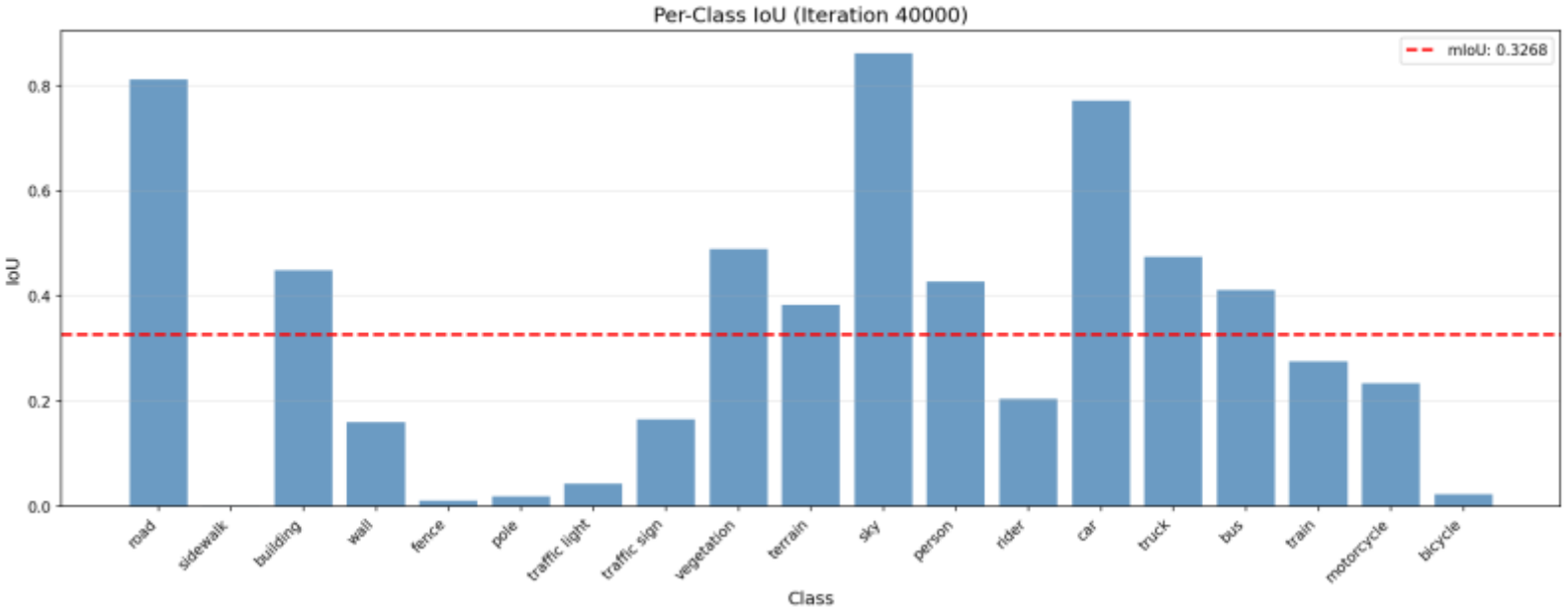
Pseudo Ratio 曲线



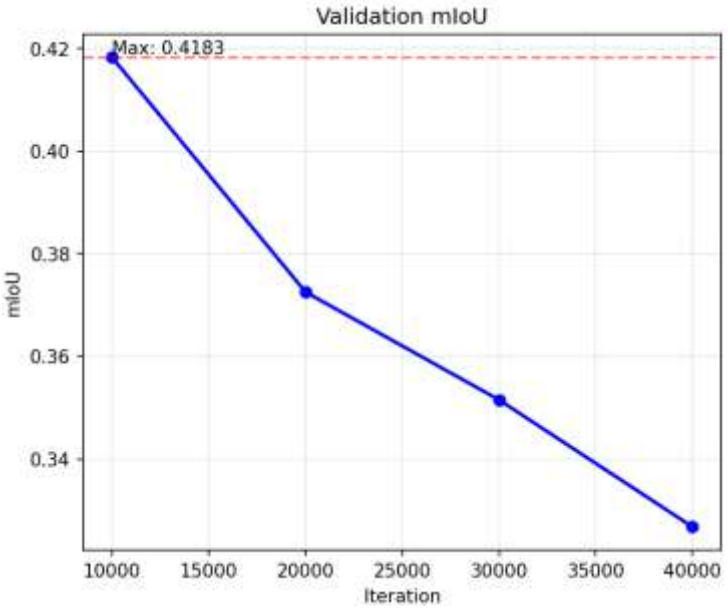
LR 曲线

这三条曲线相对正常：Total Loss 曲线呈降低趋势，趋于收敛，但抖动较频繁；Pseudo Ratio 随训练的进行成升高趋势，这源于教师模型性能的提升，是合理的；LR 曲线与实现的带有预热功能的学习率调度器是吻合的。

03 实施细节



训练的最终结果（各类 IoU）



四次 val 的 mIoU 变化

但模型最终的性能**未达到预期**，低于在第二阶段 1/4 数据集上训练的结果（0.596 mIoU），低于官方在完整数据集训练的结果（0.683 mIoU）。两个**异常点**：一是 **sidewalk** 的特征几乎没有学习到，二是随着训练**模型的性能越来越差** —— 我把原因归结到**断点续训**的不完整实现上：查阅资料，已知的局限有 Optimizer momentum/state 无法被保存和恢复，更何况我进行了**数 20 次续训**。

04 总结反思

04 总结反思

4.1 回顾

时间	做了什么	收获 / 困难
9 – 11 月	1. 阅读并用 CPU 复现了 FCN，领域入门 2. 四篇论文的阅读与汇报	入门了 CV，实现了诸多第一次：训练模型、做论文笔记、PPT 制作与汇报；遗憾是论文选择过于草率。
12 月	DAFormer 的 Pytorch 复现	第一次用服务器训练模型，在论文选择、数据获取、环境配置、CPU OOM 问题解决上花费较多时间。
期末前	另四篇论文的阅读与汇报	完成较为顺利
期末后至今	1. 系统学习了 Jittor 和 Pytorch 2. DAFormer 的 Jittor 迁移	对模型代码有了系统且清晰的理解，困难在于 GPU OOM 问题的解决。

04 总结反思

4.2 反思

收获

- 通过实践，对“科研在做什么”有了切身的体会和清晰的感知。
- 感受到了陡峭的学习曲线，在不断的试错中提升了解决问题的能力。

不足

- 受时间和硬件条件等限制，最终的复现并未达到理想的结果。
- 部分问题的处理较为笨拙，后续仍需加强交流、持续学习。

恳请老师批评与指正

南开大学 计算机科学与技术

2023级 付炘浩